

Pengertian

Round-Robin Scheduling (RRS) merupakan salah satu **algoritma scheduling pada CPU dimana semua proses yang dijalankan oleh algoritma ini akan dieksekusi secara Cyclic**. Dengan kata lain, **algoritma ini akan menjalankan suatu proses dalam batas waktu tertentu dan apabila proses tersebut telah berjalan melewati batas waktu yang ditentukan, maka proses ini akan otomatis diberhentikan sementara (*pause*) dan dimasukkan ke dalam antrian proses (*queue*) yang paling belakang (kemudian algoritma ini akan lanjut menjalankan proses lain dari *queue* yang paling depan)**. Round-Robin adalah salah satu algoritma scheduling yang paling tua, mudah diterapkan, serta paling sederhana untuk digunakan. Hal tersebutlah yang menyebabkan RRS Algorithm banyak digunakan untuk menjalankan berbagai proses secara multitasking. Selain itu, algoritma Round-Robin juga bersifat starvation-free, yang berarti algoritma ini dapat menjalankan semua proses secara bergantian tanpa memprioritaskan proses-proses tertentu (low priority process tidak diblokir dan high priority process tidak akan terus terexecute). Selain untuk menjalankan suatu proses, Round-Robin Scheduling juga dapat digunakan untuk menangani masalah lain pada sistem operasi, seperti scheduling terhadap paket data pada jaringan komputer dan sebagainya.

Konsep Kerja

Seperti yang telah dijelaskan sebelumnya, algoritma Round-Robin Scheduling bekerja dengan menggunakan sistem *time sharing* atau pembagian waktu secara adil pada setiap proses yang akan dieksekusi atau dijalankan, yakni dengan menggunakan *static quantum time* (setiap proses memperoleh $\frac{1}{n}$ waktu keseluruhan CPU). Hal ini berarti algoritma ini akan mengeksekusi atau menjalankan suatu proses berdasarkan *quantum time* yang telah diberikan. Apabila waktu eksekusi tersebut telah habis, maka algoritma ini akan berhenti menjalankan proses tersebut dan memasukkannya pada *queue* terakhir. Kemudian, algoritma ini akan melanjutkan eksekusi pada proses lain yang berada pada antrian (*queue*) paling depan berdasarkan *quantum time* yang diberikan dengan urutan kerja yang sama dengan proses sebelumnya. Singkatnya, setiap proses akan mengalami eksekusi dan *pause* yang sama berdasarkan *quantum time* yang diberikan serta tidak ada prioritas tertentu dalam algoritma ini.

Konsep kerja dari algoritma Round-Robin ini tentunya sangat bergantung pada *quantum time* yang telah ditentukan (umumnya antara 10 – 100 milisekon). Hal ini dikarenakan apabila *quantum time* yang diberikan terlalu kecil, maka akan menimbulkan adanya *context switching* yang lebih besar pada proses sehingga dapat menyebabkan terjadinya *overhead* pada CPU. Sebaliknya, apabila *quantum time* terlalu besar, maka *response time* dari proses-proses yang terkait akan semakin tinggi.

Contoh Pengaplikasian Algoritma di Dalam OS Komputer

Berikut contoh implementasi algoritma Round-Robin dalam Operating System: Sebuah CPU dengan quantum 4 mendapat beberapa proses dengan waktu kedatangan yang bersamaan seperti berikut:

Proses	Burst time
P1	4
P2	9
P3	6
P4	5
P5	3

Langkah Pertama:

Membuat Gantt Chart

Masukkan proses yang berurutan, kemudian kurangi Burst Time dengan nilai quantum.

Proses	Burst time
P1	4
P2	9
P3	6
P4	5
P5	3

Quantum= 4

P1:

Burst time – Quantum $4 - 4 = 0$.

Waktu = 4

P2:

Burst time – Quantum $9 - 4 = 5$.

Sisa = 5

Waktu = 4

P3:

Burst time – Quantum $6 - 4 = 2$.

Sisa = 2

Waktu = 4

P4:

Burst time – Quantum $5 - 4 = 1$.

Sisa = 1

Waktu antrian = 4

P5:

Burst time – Quantum

$3 - 4 = (-1)$.

Waktu antrian = 3

Proses yang memiliki sisa, kemudian dimasukkan kembali kedalam antrian sesuai dengan urutan Prosesnya.

P2:

Sisa burst time – Quantum $5 - 4 = 1$

Sisa = 1

Waktu antrian = 4

P3:

Sisa Burst time – Quantum $2 - 4 = (-2)$

Waktu antrian = 2

P4:

Sisa Burst time – Quantum $1 - 4 = (-3)$

Waktu antrian = 1

P2:

Sisa Burst time – Quantum $1 - 4 = (-3)$

Waktu antrian = 1

P1	P2	P3	P4	P5	P2	P3	P4	P2	
0	4	8	12	16	19	23	25	26	27

Langkah Kedua:

Menghitung AWT (Average Waiting Time)

Proses	Waiting Time
P1	0 (Karena P1 adalah proses yang pertama, maka waiting time-nya 0)
P2	$4 + (19 - 8) + (26 - 23) = 18$
P3	$8 + (23 - 12) = 19$
P4	$12 + (25 - 16) = 21$
P5	16

AWT yang terjadi adalah: $(0 + 18 + 19 + 21 + 16) / 5 = 74 / 5 = 14,8$

Langkah Ketiga:

Menghitung ATAT (Average Turn Around Time)

Proses	<u>Saat Tiba</u>	Lama Proses	<u>Saat Mulai</u>	<u>Saat Selesai</u>	Waiting Time	<u>Turn Around Time</u>
P1	0	4	0	4	0	4
P2	0	9	4	27	18	27
P3	0	6	8	25	19	25
P4	0	5	12	26	21	26
P5	0	3	16	19	16	19
					<u>Jumlah</u> Rata-rata	101 20,2

Dari proses tersebut, dapat diketahui nilai ATAT dari CPU tersebut adalah 20,2.